

C++ 스마트 포인터와 메모리 관리: reference counting와 JVM 가비지 컬렉션 비교 분석 보고서

1. 서론

C++는 고성능과 세밀한 제어를 제공하는 언어이지만, 수동 메모리 관리라는 본질적 문제를 안고 있다. 개발자는 `new` 로 할당한 메모리를 `delete` 로 반드시 해제해야 한다. 이 과정을 놓치면 **메모리 누수(memory leak)**가 발생하고, 이미 해제된 영역을 다시 사용하는 **dangling pointer** 문제가 발생한다. 또한 동일 메모리를 두 번 해제하면 프로그램이 불안정해진다. 이러한 오류는 코드가 복잡해질수록 발생 확률이 높다. 이처럼 메모리 관리의 어려움 때문에 C++11부터 도입된 **스마트 포인터(smart pointer)**가 큰 주목을 받았다. 스마트 포인터는 객체의 소유권을 명확히 하고 소멸자에서 메모리를 자동으로 반환하여 개발자의 부담을 줄인다. 본 보고서는 스마트 포인터의 개념과 필요성을 설명하고, `shared_ptr` 에서 사용하는 **reference counting**의 원리와 한계를 분석한다. 또한 Java Virtual Machine(JVM)의 가비지 컬렉션과 비교하여 C++의 자동 메모리 관리가 갖는 장단점을 비판적으로 논의한다.

2. 스마트 포인터의 개념과 필요성

2.1 일반 포인터 사용 시 발생하는 문제

C++ 프로그래머는 동적 메모리를 직접 관리해야 한다. 다음과 같은 문제점이 자주 발생한다.

1. **메모리 누수** – 할당한 메모리를 해제하지 않으면 메모리가 계속 점유되어 프로그램의 성능과 안정성을 저하시킨다. C++은 자동 가비지 컬렉션을 지원하지 않으므로 이런 누수가 쉽게 발생한다.
2. **Dangling pointer** – 이미 해제된 메모리 주소를 여전히 가리키는 포인터가 있는 경우 예측 불가능한 동작을 야기한다. 메모리를 해제한 후 포인터를 초기화하지 않으면 이런 문제가 발생한다.
3. **Double deletion** – 동일한 메모리를 두 번 `delete` 하는 경우 런타임 오류가 발생한다. 소유권을 명확히 관리하지 않으면 여러 포인터가 같은 객체를 삭제하는 일이 생긴다.

이러한 오류는 코드 유지 비용을 증가시키고 시스템 안정성을 해친다. 특히 예외 상황이나 복잡한 제어 흐름에서 메모리 해제를 놓치기 쉽다. 따라서 메모리 관리 부담을 줄이는 자동

화된 기법이 필요하다.

2.2 스마트 포인터란 무엇인가?

스마트 포인터는 RAII(Resource Acquisition Is Initialization) 원칙을 구현한 객체로, 내부에 원시 포인터를 저장하지만 스코프를 벗어날 때 소멸자에서 자동으로 메모리를 반환한다. GeeksforGeeks는 스마트 포인터를 “스택에 배치되는 객체로서 원시 포인터를 래핑하여 메모리 누수, dangling pointer, double deletion을 예방한다”라고 설명한다. RAII는 객체의 생명주기에 자원 획득과 해제를 묶어 **지역화된 가비지 컬렉터** 역할을 한다. 따라서 예외나 조기 반환이 발생해도 소멸자가 호출되어 안전하게 자원이 정리된다.

2.3 스마트 포인터의 유형과 특징

C++ 표준 라이브러리는 다양한 스마트 포인터를 제공한다. 소유권 규칙에 따라 주요 포인터를 비교하면 다음과 같다.

스마트 포인터	주요 특징	용도
<code>std::unique_ptr<T></code>	단독 소유. 복사할 수 없고 이동 (<code>std::move</code>)만 가능하며 소멸자에서 자동 해제된다.	다른 곳에서 공유되지 않는 자원을 관리할 때 적합. 컨테이너에 소유권을 전달하는 데 유용.
<code>std::shared_ptr<T></code>	여러 포인터가 같은 객체를 공유하는 참조 카운트 방식. 카운트가 0이 되면 객체가 파괴된다.	여러 모듈에서 같은 자원을 공유할 필요가 있을 때 사용.
<code>std::weak_ptr<T></code>	<code>shared_ptr</code> 와 달리 카운트를 증가시키지 않는 비소유 참조. 대상이 파괴됐을 때 이를 감지할 수 있다.	순환 참조를 방지하거나 상태 감시 목적으로 사용.

스마트 포인터는 메모리 해제를 자동화하여 수동 관리의 번거로움을 줄이고, 예외 안전성을 향상시킨다. 하지만 `shared_ptr`는 참조 카운트 유지에 따른 오버헤드가 있으며 순환 참조 문제를 해결하려면 `weak_ptr` 등을 조합해야 한다.

3. reference counting의 원리

3.1 reference counting 정의

reference counting은 객체를 참조하는 포인터의 수를 기록하여 더 이상 참조되지 않으면 즉시 메모리를 해제하는 기법이다. Wikipedia에서는 reference counting을 “참조가 사라지는 즉시 회수하여 긴 정지 시간을 피할 수 있는 방식”이라고 설명한다. C++의 `shared_ptr`는 이러한 reference counting을 구현한다. 카운트는 원자적으로 증가·감소해야 하므로 멀티스레드 환경에서는 추가 비용이 발생한다.

3.2 `shared_ptr`의 동작 원리

`shared_ptr`의 핵심은 **제어 블록(control block)**에 있다. 제어 블록은 실제 객체에 대한 포인터와 **소유 참조 카운트(use count)**, **약한 참조 카운트(weak count)**를 저장한다. Educated Guesswork의 분석에 따르면, `shared_ptr`를 복사하면 제어 블록의 카운트가 1 증가하고, 소멸자에서 감소하여 0이 되면 객체와 제어 블록을 함께 파괴한다. 예를 들어:

```
std::shared_ptr<int>p1 = std::make_shared<int>(10);
std::shared_ptr<int>p2 =p1;// 카운트: 2
{
    std::shared_ptr<int>p3 =p1;// 카운트: 3
} // p3 소멸 → 카운트: 2
p2.reset();// p2가 소유권 해제 → 카운트: 1
// p1이 스코프를 벗어나면 카운트가 0이 되어 객체가 해제됨
```

참조 카운트 방식의 핵심은 카운트가 0이 되는 시점이 곧 객체의 파괴 시점이라는 점이다. 이로써 객체 수명이 명확하게 정의되어 예측 가능한 메모리 회수가 가능하다.

3.3 장점과 한계

reference counting 방식은 다음과 같은 장점이 있다.

- **즉각적 회수** – 객체가 더 이상 사용되지 않으면 바로 해제되어 긴 정지 시간 없이 메모리를 회수한다.
- **명확한 생명주기** – 객체의 파괴 시점이 정확하게 결정되어 파일 핸들, 네트워크 소켓 등 비메모리 자원 관리에 적합하다.
- **간단한 구현** – 알고리즘이 단순하여 임베디드 시스템 등에서도 구현이 가능하다.

그러나 한계도 명확하다.

- **성능 오버헤드** – 포인터 복사와 해제 시마다 카운트를 증가·감소해야 하며, 멀티스레드에서는 원자적 연산을 사용해 비용이 늘어난다.
- **순환 참조 문제** – 서로를 참조하는 두 객체가 `shared_ptr`를 통해 연결되어 있으면 카운트가 0이 되지 않아 메모리가 해제되지 않는다. 이를 해결하려면 한쪽을 `weak_ptr`로 선언해 소유 관계를 끊어야 한다.
- **캐시 지역성 저하** – 제어 블록이 별도 메모리 공간에 존재하여 캐시 효율이 떨어질 수 있다.

4. Java Virtual Machine의 메모리 관리와 비교

4.1 JVM의 메모리 관리

Java 프로그램은 JVM 위에서 실행되며, 메모리 관리는 **가비지 컬렉션(GC)** 이 담당한다. GeeksforGeeks는 Java 메모리를 스택과 힙으로 구분한다: **스택**에는 메서드 호출 정보, 기본 타입 지역 변수 및 객체 참조가 저장되고, **힙**에는 `new` 로 생성된 객체가 저장된다. 스택 메모리는 메서드 종료 시 자동으로 반환되지만, 힙에 있는 객체는 더 이상 참조되지 않을 때 GC가 이를 삭제한다.

GC는 대표적으로 **mark-and-sweep** 알고리즘을 사용한다. New Relic의 설명에 따르면 GC는 실행 중인 모든 객체 그래프를 따라가면서 reachable 객체를 표시(mark)한 뒤, 표시되지 않은(unreachable) 객체를 수거(sweep)하여 메모리를 회수한다. Java의 힙은 **Young Generation**과 **Old Generation**으로 구분되어 세대별 GC를 수행한다. Aerospike는 GC가 메모리 누수와 dangling pointer를 감소시키지만, 힙 전체를 검사하는 과정에서 프로그램이 일시적으로 중단되는 **정지 시간(stop-the-world pause)** 을 야기한다고 지적한다. GC의 실행 시점과 지속 시간은 예측하기 어려워 실시간 시스템에 부적합할 수 있다.

4.2 공통점과 차이점

공통점

- **자동 메모리 반환** – C++ 스마트 포인터와 JVM GC 모두 개발자가 직접 `delete` 를 호출하지 않아도 메모리를 자동으로 회수한다. 스마트 포인터의 소멸자 혹은 GC가 해당 책임을 수행한다. 이로 인해 메모리 누수와 dangling pointer를 줄일 수 있다.
- **개발 생산성 향상** – 메모리 해제 코드를 명시하지 않아도 되므로 로직 작성에 집중할 수 있다. RAII는 지역적 가비지 컬렉터처럼 동작하며 예외 상황에서도 안전하다.

차이점

구분	C++ 스마트 포인터 (RAII/Reference Counting)	JVM 가비지 컬렉션
생명주기 결정	객체가 소유자 스코프를 벗어나거나 reference count가 0이 되는 순간 즉시 해제된다. 따라서 파괴 시점이 결정적(deterministic) 이다.	GC는 주기적으로 힙을 스캔하여 도달할 수 없는 객체를 찾아 정리한다. 삭제 시점이 명시되지 않으므로 비결정적(non-deterministic) 이다.
오버헤드	<code>shared_ptr</code> 는 포인터 복사 시 카운트 업데이트가 필요하고, 멀티스레드에서는 원자적 연산이 요구돼 성능 비용이 있다. 또한 순환 참조 문제를 수동으로 관리해야 한다.	GC는 힙 전체를 탐색하는 동안 애플리케이션을 멈추는 정지 시간이 발생하고, GC 스레드가 추가 CPU를 사용한다. 그러나 개발자는 순환 참조를 별도로 관리하지 않아도 된다.
실시간성	reference counting은 즉각적으로 메모리를 회수하므로 real-time 환경에서 예측 가능하다. 그러나 카운트 업	GC는 정지 시간이 존재하므로 하드 real-time 시스템에 적합하지 않다. 새로운 GC 알고리즘(Shenandoah,

구분	C++ 스마트 포인터 (RAII/Reference Counting)	JVM 가비지 컬렉션
	데이트가 빈번해 캐시 성능에 영향을 줄 수 있다.	ZGC 등)은 지연을 줄이려 하지만 완전히 제거하긴 어렵다.
순환 참조	순환 구조에서는 카운트가 0이 되지 않아 메모리가 해제되지 않는다. 이를 해결하기 위해 <code>weak_ptr</code> 를 사용해 비소유 관계를 만들어야 한다.	GC는 그래프 탐색을 통해 순환 참조를 자동으로 감지하고 해제할 수 있다.
자원 외 메모리 관리	RAII는 파일 핸들, 소켓 등 메모리 외 자원에도 적용 가능하여 스코프 종료 시 자동으로 닫는다.	GC는 메모리 회수만 다루며 비메모리 자원은 개발자가 명시적으로 해제해야 한다.

4.3 개발자 관점에서 의미

C++과 Java의 자동 메모리 관리는 서로 다른 철학을 가진다. C++은 성능과 예측 가능성을 중시하여 개발자에게 더 많은 제어권을 제공한다. RAII와 스마트 포인터는 자원 해제를 코드 구조와 스코프에 묶어버림으로써 안전성과 성능을 확보하되, 순환 참조와 카운트 관리 비용을 개발자에게 떠넘긴다. 반면 Java의 GC는 메모리 관리의 복잡성을 숨기고 개발자가 로직에 집중하도록 돕지만, GC가 언제 실행될지 예측하기 어렵고 성능 튜닝을 위해 다양한 GC 정책을 고려해야 한다. 따라서 두 방식은 서로 보완적인 관점을 제공하며, 개발자는 사용 환경에 맞는 메모리 관리 전략을 선택해야 한다.

5. 메모리 자동 반환 방식의 장단점

5.1 장점

- **코드 안정성 향상** – 메모리 해제를 자동으로 수행함으로써 누수와 dangling pointer를 줄여 프로그램 안정성을 높인다.
- **예외 안전성** – 예외가 발생하거나 조기 반환이 있어도 소멸자가 실행되어 자원이 정리된다.
- **개발 생산성** – 메모리 해제 코드 작성과 디버깅 부담이 감소한다. 스마트 포인터는 소유권 규칙을 명확히 하여 코드 이해도를 높인다.
- **정확한 자원 회수** – reference counting은 객체의 참조가 모두 사라지는 즉시 자원을 반환하므로, 리소스 대기 시간이 짧다.

5.2 단점

- **성능 오버헤드** – `shared_ptr` 는 카운트 업데이트를 위해 추가 연산과 메모리가 필요하며, 멀티스레드 환경에서는 원자적 연산으로 비용이 증가한다.

- **순환 참조 문제** – reference counting은 순환 구조를 자동으로 해제하지 못해 메모리 누수가 발생한다. 이를 방지하려면 설계를 조심하거나 `weak_ptr` 를 사용해야 한다.
- **제어의 제약** – 자동 반환 방식에서는 메모리 해제 시점이 알고리즘에 묶여 있어, 특정 시점에 미리 메모리를 반환하려면 특별한 조치를 취해야 한다. GC 환경에서는 명시적으로 객체를 파괴할 수 없고 시스템이 회수하는 시점을 기다려야 한다.
- **런타임 오버헤드** – GC는 힙을 스캔하는 데 시간이 필요하며 프로그램을 일시 정지시킨다. reference counting은 카운트 연산과 제어 블록 할당으로 인한 캐시 미스가 발생한다.

5.3 자동 반환 방식에 대한 의견

자동 메모리 반환은 메모리 관리 오류를 크게 줄이는 효과적 수단이지만, **모든 상황에 최선은 아니다**. C++에서 `unique_ptr` 와 `shared_ptr` 는 자원 관리의 표준이 되었으나, 참조 카운트의 성능 오버헤드와 순환 참조 문제를 고려해야 한다. 성능이 중요한 영역에서는 `unique_ptr` 나 객체 풀 같은 대안을 검토할 수 있다. Java의 GC는 프로그래머의 부담을 줄이지만, 실시간성 요구가 높은 시스템에는 적합하지 않으며, GC 튜닝과 메모리 프로파일링이 필요하다. 요약하면, 자동 메모리 반환은 **안전성과 생산성을 높이는 강력한 도구**이지만, 언어와 애플리케이션 특성에 맞춰 적절히 사용해야 한다.

6. 결론

본 보고서는 C++ 스마트 포인터와 reference counting의 원리를 분석하고, Java Virtual Machine의 가비지 컬렉션과 비교하여 자동 메모리 관리의 장단점을 평가하였다. 스마트 포인터는 RAII 원칙을 구현하여 메모리 누수와 dangling pointer를 줄이며, 예외 안전성을 확보한다. 특히 `shared_ptr` 는 reference counting을 통해 다중 소유권을 제공하지만 순환 참조 문제와 카운트 업데이트 오버헤드가 존재한다. `weak_ptr` 를 사용해 순환 구조를 끊어야 메모리가 정상적으로 해제된다. JVM의 GC는 메모리 관리의 복잡성을 숨기고 순환 참조를 자동으로 처리하지만, 비결정적인 회수 시점과 정지 시간, 성능 오버헤드라는 단점이 있다. 두 방식은 목적과 환경에 따라 장점을 발휘하므로, 개발자는 성능 요구와 코드 안전성을 고려하여 적절한 메모리 관리 기법을 선택해야 한다.

7. 참고자료

1. **GeeksforGeeks – "C++ smart pointers"** – 스마트 포인터 정의와 필요성을 설명하고 `unique_ptr` , `shared_ptr` , `weak_ptr` 의 특징을 소개한다. 출처:
<https://www.geeksforgeeks.org/cpp/smart-pointers-cpp/>
2. **Codecademy – "Smart Pointers in C++"** – 스마트 포인터의 종류와 소유권 규칙을 설명하며, `unique_ptr` 가 복사 불가능함을 강조한다. 출처:
<https://www.codecademy.com/article/what-are-smart-pointers-in-cpp>

3. **Educated Guesswork – "Understanding Memory Management, Part 3: C++ Smart Pointers"** – `shared_ptr`의 control block 구조와 참조 카운트 증가·감소 과정을 예제로 설명한다. 출처: <https://educatedguesswork.org/posts/memory-management-3/>
4. **LearnCpp – "Circular dependency issues with std::shared_ptr and std::weak_ptr"** – 두 객체가 서로를 `shared_ptr`로 참조할 때 순환 참조로 인해 메모리가 해제되지 않는 사례를 제시한다. 출처: https://www.learncpp.com/cpp-tutorial/circular-dependency-issues-with-stdshared_ptr-and-stdweak_ptr/
5. **GeeksforGeeks – "Memory leak in C++"** – C++의 메모리 누수 문제와 자동 해제를 지원하지 않는 언어 특성을 지적한다. 출처: <https://www.geeksforgeeks.org/c/what-is-memory-leak-how-can-we-avoid/>
6. **TheCodedMessage.com – "RAII: Compile-Time Memory Management in C++ and Rust"** – RAII가 런타임 가비지 컬렉션과는 달리 컴파일 시간에 자원 해제 책임을 명시적으로 관리하며, reference counting과 가비지 컬렉션의 오버헤드를 비교한다. 출처: <https://www.thecodedmessage.com/posts/raii/>
7. **dev.to – "Memory management and RAII"** – RAII가 객체 수명에 자원을 묶어 지역화된 가비지 컬렉터처럼 동작하고, 예외 상황에서도 안전하게 해제함을 설명한다. 출처: <https://dev.to/10xlearner/memory-management-and-raii-4f20>
8. **Wikipedia – "Reference counting"** – reference counting의 장점과 단점을 조망하면서 순환 참조 문제와 멀티스레드 환경에서의 원자적 연산 비용을 언급한다. 출처: https://en.wikipedia.org/wiki/Reference_counting
9. **GeeksforGeeks – "Stack vs Heap Memory Allocation"** – Java와 C/C++의 스택과 힙 구조를 비교하고, 힙을 세대별로 나누어 GC를 수행하는 방식을 설명한다. 출처: <https://www.geeksforgeeks.org/dsa/stack-vs-heap-memory-allocation/>
10. **New Relic – "Java garbage collection: What is it and how does it work?"** – mark-and-sweep 알고리즘과 Java에서 GC가 도달 불가능한 객체를 찾고 삭제하는 과정을 설명한다. 출처: <https://newrelic.com/blog/apm/java-garbage-collection>
11. **Aerospike – "Understanding garbage collection: A developer's guide to memory management"** – GC가 메모리 누수와 dangling pointer 문제를 줄이지만 stop-the-world pause와 예측 불가능한 실행 시점을 갖는다는 단점을 언급한다. 출처: <https://aerospike.com/blog/understanding-garbage-collection/>